Hand Interaction Metrics Manual Run

Purpose

This procedure uses the original Hand-Interaction MATLAB code to produce reference hand-use metrics for comparison with EgoModelKit. It tests the same 17-second fridge session in two forms:

1. Single-video session:
    - fridge_17s.mp4
2. Multi-video session:
    - fridge_10s.mp4
    - fridge_7s.mp4

Each session is reported twice:

- Right hand dominant after injury
- Left hand dominant after injury

The resulting four reference rows are:

- Single video, right dominant
- Single video, left dominant
- Multi video, right dominant
- Multi video, left dominant

The main expected invariants are:

- The single-video and multi-video sessions produce the same session-level metrics when their source frames are identical.
- Changing the dominant hand only swaps the dominant and non-dominant values.
- Raw right-hand and left-hand values do not change with the dominance selection.
- Bilateral values do not change with the dominance selection.

Required files

Place the following files in `~/Downloads`:

Code.zip
fridge_17s.mp4
fridge_10s.mp4
fridge_7s.mp4
shan_json_to_bandini_profile.py
run_bandini_reference.m

The procedure resumes from the Shan Linux reference setup.

Outputs

The main output is:
~/bandini-reference-run/results/bandini_reference_metrics.csv

Additional traceability outputs are:
single_statepool_profile.csv
multi_statepool_profile.csv
single_final_interaction_profile.csv
multi_final_interaction_profile.csv

The raw converted contact-state profiles are retained under:
~/bandini-reference-run/profiles

The Shan predictions are retained under:
~/bandini-reference-run/shan

Exact behavior of the original MATLAB workflow

The original code performs the following processing.

Contact-state conversion

For each hand:

Contact state < 3 → no interaction
Contact state ≥ 3 → interaction

Statepool

`interactionDetectionOutput.m` divides each video into consecutive, non-overlapping 30-frame windows.

Each window is replaced by its modal value.

At 30 FPS, this represents one-second windows.

A 15-to-15 tie resolves to zero because MATLAB mode returns the smaller value.

The Statepool window restarts at the beginning of each physical video file.

For the current multi-video test this does not create a boundary difference because:

fridge_10s.mp4 = 300 frames
fridge_7s.mp4 = 210 frames

Both frame counts are exact multiples of 30.

Additional filtering

outcomeMeasures_Extraction.m then calls the original filterInteractionOutput.m, which applies:

1. A causal 120-frame moving average.
2. Min-max normalization.
3. A strict threshold greater than 0.5.

This second filtering stage is separate from 30-frame Statepool.

Time calculation

The original extraction script uses:
0:0.033:0.033*(frame_count-1)

It therefore uses 0.033 seconds per frame rather than exactly 1/30 seconds.

For 510 frames, its recorded duration is:
509 × 0.033 = 16.797 seconds

This value is preserved in the legacy reference output.

Metrics

For each physical hand:

Perc = interacting frames / total frames
Dur = mean duration of the detected interaction segments
Num = number of interaction segments / recording hours

Perc is stored as a fraction from 0 to 1, not as a number from 0 to 100.

Dominance mapping

The original extraction code produces right-hand and left-hand values. Dominant and non-dominant labels are applied afterward.

Right-dominant run:
Dominant   = right
Non-dominant = left

Left-dominant run:
Dominant   = left
Non-dominant = right

Bilateral values

Use the formulas from the paper and outcomeMeasures_Validation_dominant.m:

Perc bilateral = (Perc dominant + Perc non-dominant) / 2
Dur bilateral = Dur dominant + Dur non-dominant
Num bilateral = Num dominant + Num non-dominant

Do not use the separate OR-profile calculation in the local calcOutcomeMeasures_bi function as the primary comparison with EgoModelKit. It represents a different bilateral calculation from the formulas used in the paper and dominant-hand validation script.

Why a small external harness is required

The supplied MATLAB files are research dataset scripts rather than reusable command-line programs.

They contain:

- Hard-coded Windows drive paths.
- Participant-folder discovery.
- Fixed character positions for participant IDs and recording dates.
- Dataset-specific exclusions.
- No command-line parameters.
- No direct CSV export of the four test configurations.

The supplied helper files therefore:

- Leave every original MATLAB file unchanged.
- Read the Shan JSON outputs produced by the existing reference detector.
- Expose the original Statepool and metric-extraction logic for the fridge tests.
- Call the original filterInteractionOutput.m unchanged.
- Add dominance mapping and stable CSV output.
- Preserve all source frames, assigning state zero when no hand is detected.

These are test-harness and interoperability changes. They do not alter the Shan detector or the original filterInteractionOutput.m.

1. Verify the required tools

Confirm that the required Linux tools are installed:

```
command -v git unzip ffmpeg ffprobe sha256sum
command -v matlab
```

Each command should print a path.

Install missing open-source utilities when administrator access is available:

```
sudo apt update
sudo apt install -y git unzip ffmpeg
```

MATLAB must be installed and licensed separately.

Confirm the resumed Shan setup:

```
source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate handobj_new

test -d "$HOME/hoc-reference/hand_object_detector"

test -f \
"$HOME/hoc-reference/hand_object_detector/models/res101_handobj_100K/p
ascal_voc/faster_rcnn_1_8_132028.pth"
python - <<'PY'
import cv2
import torch

print("OpenCV:", cv2.__version__)
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
PY
```

Expected key result:

```
CUDA available: False
```

The reference Shan detector remains CPU-only. A GPU is not required for the metric calculation.

2. Extract and verify the original code

Create the reference workspace:

```
mkdir -p "$HOME/bandini-reference"
cd "$HOME/bandini-reference"
unzip -q "$HOME/Downloads/Code.zip"
```

Set the source path:

```
CODE_ROOT="$HOME/bandini-reference/Code"
CLINICAL_DIR="$CODE_ROOT/Matlab/ClinicalValidation"
```

3. Prepare the run directory

Create a clean run directory:

```
RUN_ROOT="$HOME/bandini-reference-run"

rm -rf "$RUN_ROOT"

mkdir -p \
  "$RUN_ROOT/inputs" \
  "$RUN_ROOT/frames" \
  "$RUN_ROOT/shan" \
  "$RUN_ROOT/profiles" \
  "$RUN_ROOT/results" \
  "$RUN_ROOT/tools"
```

Copy the videos:

```
cp "$HOME/Downloads/fridge_17s.mp4" \
  "$RUN_ROOT/inputs/"

cp "$HOME/Downloads/fridge_10s.mp4" \
  "$RUN_ROOT/inputs/"

cp "$HOME/Downloads/fridge_7s.mp4" \
  "$RUN_ROOT/inputs/"

Copy the helper files:

cp "$HOME/Downloads/shan_json_to_bandini_profile.py" \
  "$RUN_ROOT/tools/"

cp "$HOME/Downloads/run_bandini_reference.m" \
  "$RUN_ROOT/tools/"

chmod +x \
  "$RUN_ROOT/tools/shan_json_to_bandini_profile.py"
```

4. Inspect the source videos

Display the video metadata:

```
for video in "$RUN_ROOT"/inputs/*.mp4
do
  echo "===== $(basename "$video") ====="

  ffprobe \
    -v error \
    -select_streams v:0 \
    -show_entries \

stream=width,height,avg_frame_rate,r_frame_rate,nb_frames,duration \
    -of default=noprint_wrappers=1 \
    "$video"
done
```

The intended test inputs represent:
fridge_17s.mp4 = 17 seconds
fridge_10s.mp4 = first 10 seconds
fridge_7s.mp4 = remaining 7 seconds

5. Extract the model frames

Activate the established Shan environment:
source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate handobj_new

Define a reusable extraction function:

```
extract_reference_frames() {
  input_video="$1"
  video_stem="$2"
  output_dir="$RUN_ROOT/frames/$video_stem"

  mkdir -p "$output_dir"

  ffmpeg \
    -hide_banner \
```

```
    -loglevel error \
    -y \
    -i "$input_video" \
    -vf "fps=30" \
    "$output_dir/${video_stem}_%06d.png"

  python - "$output_dir" <<'PY'
from pathlib import Path
import sys
import cv2

frame_dir = Path(sys.argv[1])

for path in sorted(frame_dir.glob("*.png")):
    image = cv2.imread(str(path))

    if image is None:
        raise RuntimeError(f"Could not decode {path}")

    resized = cv2.resize(
        image,
        (720, 405),
        interpolation=cv2.INTER_LINEAR,
    )

    if not cv2.imwrite(str(path), resized):
        raise RuntimeError(f"Could not write {path}")

print("Resized frames:", len(list(frame_dir.glob("*.png"))))
PY
}
```

Extract all three inputs:

```
extract_reference_frames \
  "$RUN_ROOT/inputs/fridge_17s.mp4" \
  "fridge_17s"

extract_reference_frames \
  "$RUN_ROOT/inputs/fridge_10s.mp4" \
  "fridge_10s"
```

```
extract_reference_frames \
  "$RUN_ROOT/inputs/fridge_7s.mp4" \
  "fridge_7s"
```

Count the frames:

```
for stem in fridge_17s fridge_10s fridge_7s
do
  printf '%s: ' "$stem"

  find "$RUN_ROOT/frames/$stem" \
    -maxdepth 1 \
    -type f \
    -name '*.png' \
    | wc -l
done
```

Expected for exact 30 FPS test clips:
fridge_17s: 512
fridge_10s: 300
fridge_7s: 212

6. Verify that the full and split sessions contain the same frames

Create ordered frame-hash lists:

```
find "$RUN_ROOT/frames/fridge_17s" \
  -maxdepth 1 \
  -type f \
  -name '*.png' \
  -print0 \
  | sort -z \
  | xargs -0 sha256sum \
  | awk '{print $1}' \
  > "$RUN_ROOT/full_frame_hashes.txt"
```

Create the corresponding split-session list in session order:

```
{
  find "$RUN_ROOT/frames/fridge_10s" \
    -maxdepth 1 \
```

```
    -type f \
    -name '*.png' \
    -print0 \
    | sort -z \
    | xargs -0 sha256sum \
    | awk '{print $1}'

  find "$RUN_ROOT/frames/fridge_7s" \
    -maxdepth 1 \
    -type f \
    -name '*.png' \
    -print0 \
    | sort -z \
    | xargs -0 sha256sum \
    | awk '{print $1}'
} > "$RUN_ROOT/split_frame_hashes.txt"
```

Compare them:

```
diff -u \
  "$RUN_ROOT/full_frame_hashes.txt" \
  "$RUN_ROOT/split_frame_hashes.txt"
```

Expected:
No output

Also confirm equal counts:

test \

```
  "$(wc -l < "$RUN_ROOT/full_frame_hashes.txt")" \
  -eq \
  "$(wc -l < "$RUN_ROOT/split_frame_hashes.txt")"
```

7. Run the Shan detector

Set the detector path:

```
HOC="$HOME/hoc-reference/hand_object_detector"
```

Run all three frame directories:

```
source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate handobj_new

cd "$HOC"

for stem in fridge_17s fridge_10s fridge_7s
do
  mkdir -p "$RUN_ROOT/shan/$stem"

  python demo.py \
    --dataset pascal_voc \
    --cfg cfgs/res101.yml \
    --net res101 \
    --load_dir models \
    --image_dir "$RUN_ROOT/frames/$stem" \
    --save_dir "$RUN_ROOT/shan/$stem" \
    --checksession 1 \
    --checkepoch 8 \
    --checkpoint 132028 \
    --thresh_hand 0.5 \
    --thresh_obj 0.5
done
```

Count the JSON predictions:|

```
for stem in fridge_17s fridge_10s fridge_7s
do
  printf '%s JSON files: ' "$stem"

  find "$RUN_ROOT/shan/$stem" \
    -maxdepth 1 \
    -type f \
    -name '*_shan.json' \
    | wc -l
done
```

Expected:
fridge_17s JSON files: 512
fridge_10s JSON files: 300
fridge_7s JSON files: 212

8. Convert Shan predictions into raw contact-state profiles

Run the supplied adapter:

```
python "$RUN_ROOT/tools/shan_json_to_bandini_profile.py" \
  --input-dir "$RUN_ROOT/shan/fridge_17s" \
  --output-csv "$RUN_ROOT/profiles/fridge_17s_raw.csv" \
  --expected-count 512

python "$RUN_ROOT/tools/shan_json_to_bandini_profile.py" \
  --input-dir "$RUN_ROOT/shan/fridge_10s" \
  --output-csv "$RUN_ROOT/profiles/fridge_10s_raw.csv" \
  --expected-count 300

python "$RUN_ROOT/tools/shan_json_to_bandini_profile.py" \
  --input-dir "$RUN_ROOT/shan/fridge_7s" \
  --output-csv "$RUN_ROOT/profiles/fridge_7s_raw.csv" \
  --expected-count 212
```

Each profile contains:
Frame_number
ContactState_R
ContactState_L

The adapter follows the hand-side and duplicate-hand selection behavior of the original `filterHandTable` logic.

Inspect the first rows:

```
head \
  "$RUN_ROOT/profiles/fridge_17s_raw.csv"
```

9. Run the original MATLAB metric logic

Set the paths:

```
CODE_ROOT="$HOME/bandini-reference/Code"
CLINICAL_DIR="$CODE_ROOT/Matlab/ClinicalValidation"
```

Run the helper through MATLAB:

```
matlab -batch \
  "addpath('$RUN_ROOT/tools');
run_bandini_reference('$RUN_ROOT','$CLINICAL_DIR')"
```

The helper directly calls the unchanged original:

```
filterInteractionOutput.m
```

It exposes the Statepool and metric functions embedded as local functions in the original dataset scripts.

Expected final message:

```
Reference rows written: .../bandini_reference_metrics.csv
```

10. Confirm the reference outputs

List the files:

```
find "$RUN_ROOT/results" \
  -maxdepth 1 \
  -type f \
  -printf '%f\n' \
  | sort
```

Expected:
bandini_reference_metrics.csv
multi_final_interaction_profile.csv
multi_statepool_profile.csv
single_final_interaction_profile.csv
single_statepool_profile.csv

Display the metric rows:

```
column -s, -t \
  < "$RUN_ROOT/results/bandini_reference_metrics.csv"
```

The CSV contains four rows:
single-video / right dominant
single-video / left dominant
multi-video / right dominant
multi-video / left dominant

It also preserves:

- Raw right-hand metrics
- Raw left-hand metrics
- Dominant-hand metrics
- Non-dominant-hand metrics
- Bilateral metrics
- Legacy frame count
- Legacy recording duration
- Legacy time step